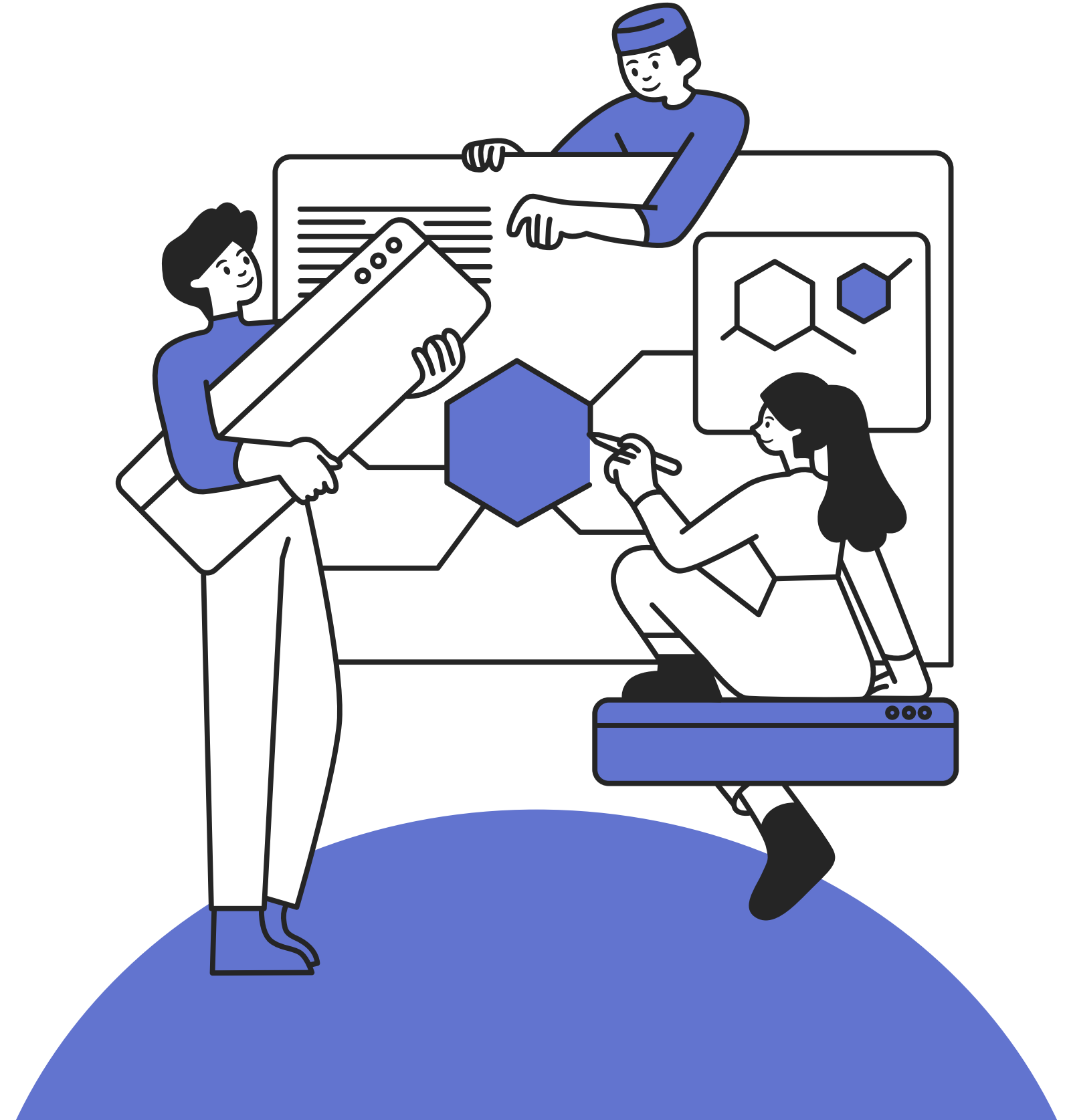


2025.10.01

프로젝트 포트폴리오

임현성



CONTENTS

- 1 프로젝트 개요
- 2 데이터 소개
- 3 프로젝트 수행 과정

- 4 결과
- 5 성과 및 배운점
- 6 추가 사항

01

프로젝트 개요

AI 기반 텍스트 분석 시스템

본 프로젝트는 네이버 스토어 상품 리뷰 데이터를 자동으로 수집하여 감성(긍정/부정)을 분석하는 AI 기반 텍스트 분석 시스템입니다. 방대한 리뷰 데이터를 효율적으로 수집하여 감성 분석을 통해 고객의 만족도와 불만 요소를 체계적으로 정리할 수 있습니다. 이는 판매자 및 마케터가 고객 피드백을 효과적으로 반영하여 제품 개선 및 전략 수립에 활용할 수 있도록 돕는 것을 목표로 합니다.



네이버 스토어 리뷰

- 본 데이터는 직접 크롤링한 네이버 스토어 리뷰입니다.
텍스트는 전처리 후 KoELECTRA로 감성 라벨을 예측해 사
용했습니다. 이 데이터로 감성 분포를 분석했고 날짜 데이터도
함께 가져와 시계열/추세 분석했습니다.

	jaso***** · 25.09.29. 신고 디자인 마음에 들어요 · 사이즈 정사이즈예요 정말 이빠요 만족합니다 0	
	★★★★★ 5 jaso***** · 25.09.29. 신고 디자인 마음에 들어요 · 사이즈 정사이즈예요 정말 귀여워요...드디어 사다니... 0	
	★★★★★ 5 ping*** · 25.09.29. 신고 디자인 화면과 같아요 · 사이즈 정사이즈예요 운 좋게 구매했어요 너무 좋아요~~~ 시크릿은 안나왔지만 마카롱이 짭이쁜거 같아요 0	
	★★★★★ 5 meot***** · 25.09.29. 신고 0	
7	정말 만족합니다! 귀여워요!!!	25.09.29.
8	정말 이빠요 만족합니다	25.09.29.
9	정말 귀여워요...드디어 사다니...	25.09.29.
10	운 좋게 구매했어요 너무 좋아요~~~ 시크릿은 안나왔지만 마카롱이 짭이쁜거 같아요	25.09.29.
11	라부부 정말 귀엽네요!!	25.09.29.
12	각각 4개 시켰는데 그 중 하나가 시크릿이었어요 너무 귀여워요~~	25.09.29.
13	각각 4개 시켰는데 그 중 하나가 시크릿이었어요 너무 귀여워요~~	25.09.29.
14	각각 4개 시켰는데 그 중 하나가 시크릿이었어요 너무 귀여워요~~	25.09.29.
15	각각 4개 시켰는데 그 중 하나가 시크릿이었어요 너무 귀여워요~~	25.09.29.
16	엄청 만족합니다~~~	25.09.29.
17	드디어 마카롱 까찌 얻었네요!!	25.09.29.
18	굿굿 조공용으로 샀어요	25.09.29.
19	라부부 항상 이빠요 좋아요	25.09.29.

03

프로젝트 수행 과정

01

데이터 수집

Python

Pandas

Selenium

02

데이터 전처리

이모지 제거

특수문자 제거

반복 자음/모음 축소

줄바꿈 제거

03

감성분석

토큰화(Tokenization)

모델 추론(KoELECTRA)

확률 계산(Softmax)

04

시각화

시각화

워드클라우드

시계열/추세 분석

03

프로젝트 수행 과정

01 데이터 수집

- 언어 & 환경
 - Python 기반 스크립트
 - Selenium 활용한 웹 자동화 클로링
- 웹드라이버 관리
 - webdriver_manager.chrome.ChromeDriverManager → 크롬 드라이버 자동 설치 및 버전 관리
- Selenium 기능
 - webdriver.ChromeOptions → 브라우저 옵션 설정
 - WebDriverWait + expected_conditions(EC) → 특정 요소 클릭 가능 여부 대기
 - find_elements(By.CSS_SELECTOR/XPATH) → HTML 요소탐색
- 리뷰 수집 로직
 - execute_script → 리뷰 탭 자동 클릭, 최신순 정렬 버튼 클릭

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from webdriver_manager.chrome import ChromeDriverManager
import time
import pandas as pd
import traceback

# 드라이버 설정
options = webdriver.ChromeOptions()
options.add_argument("--start-maximized")
options.add_experimental_option("detach", True) # 브라우저 종료 방지
driver = webdriver.Chrome(service=Service(ChromeDriverManager().install()), options=options)
wait = WebDriverWait(driver, 15)

# 상품 페이지 열기
url = "https://brand.naver.com/popmart/products/10643619141"
driver.get(url)
time.sleep(2)

# 리뷰 탭 클릭
try:
    review_tab = wait.until(
        EC.element_to_be_clickable((By.CSS_SELECTOR, "a[data-name='REVIEW']"))
    )
    driver.execute_script("arguments[0].click();", review_tab)
    print("리뷰 탭 클릭 완료")
except Exception as e:
    print("리뷰 탭 클릭 실패:", e)

time.sleep(1.5)

# 최신순 버튼 클릭
try:
    latest_btn = wait.until(
        EC.element_to_be_clickable((By.CSS_SELECTOR, "a[role='radio'][data-shp-contents-id=...]"))
    )
    driver.execute_script("arguments[0].click();", latest_btn)
    print("최신순 버튼 클릭 완료")
except Exception as e:
    print("최신순 버튼 클릭 실패:", e)

time.sleep(1.5)

# 스크롤 함수
def smooth_scroll_to(element, duration=0.5, steps=10):
    current_y = driver.execute_script("return window.scrollY;")
    target_y = driver.execute_script(
        "return arguments[0].getBoundingClientRect().top + window.scrollY;", element
    )
    distance = target_y - current_y
    for i in range(steps):
        driver.execute_script(f"window.scrollTo(0, {current_y + distance * (i+1)/steps});")
        time.sleep(duration / steps)
```

03

프로젝트 수행 과정

01 데이터 수집

• 리뷰 수집 로직

- execute_script → 리뷰 탭 자동 클릭, 최신순 정렬 버튼 클릭
- XPATH로 접근 경로 지정, CSS선택자 이용 → 리뷰 블록 단위로 텍스트 + 날짜 추출

• 페이지네이션 처리

- XPATH로 접근 경로 지정 → 10페이지 단위 블록 이동 시 “다음” 버튼 클릭

• 예외 처리

- try-except구문 활용 → 클릭 실패, 페이지 로딩 실패 등 로그 출력
- stop-crawling → 리뷰 없을 때 종료

• 데이터 저장

- pandas.DataFrame → 리스트 변환
- to_csv → csv파일 저장

```
max_pages = 100 # 총 수집 페이지
pages_per_block = 10 # 한 블록에 10페이지
current_page = 1
stop_crawling = False # 리뷰 없으면 종료

while current_page <= max_pages and not stop_crawling:
    for page_in_block in range(1, pages_per_block + 1):
        page_number = current_page + page_in_block - 1
        if page_number > max_pages:
            break

        print(f"\n--- {page_number}페이지 수집 시작 ---")
        time.sleep(2)

        try:
            review_blocks = driver.find_elements(By.CSS_SELECTOR, "#REVIEW div.HakaEZ2481")
            if not review_blocks:
                print(f"{page_number}페이지에 리뷰 없음 → 수집할 종료")
                stop_crawling = True
                break

            for block in review_blocks:
                # 리뷰 텍스트
                review_text = None
                try:
                    review_text = block.find_element(By.CSS_SELECTOR, "div.KqJ8Qqw882
span.MX910FZo2F").text.strip()
                except:
                    try:
                        review_text = block.find_element(By.CSS_SELECTOR,
"span.MX910FZo2F").text.strip()
                    except:
                        review_text = None

                # 날짜
                date_text = None
                try:
                    parent = block.find_element(By.XPATH,
"./ancestor::div[contains(@class, '02M37e85_1')]")
                    date_text = parent.find_element(By.CSS_SELECTOR, "div.Db9Dtnf7gY
span.MX910FZo2F").text.strip()
                except:
                    date_text = None

                if review_text:
                    review_list.append({'review': review_text, 'date': date_text})

            print(f"{page_number}페이지 리뷰 {len(review_blocks)}개 수집 완료")

        except Exception as e:
            print(f"{page_number}페이지 리뷰 수집 실패:", e)
            traceback.print_exc()

# 현재 블록 내 다음 페이지 클릭
if page_in_block < pages_per_block and page_number < max_pages and not stop_crawling:
    try:
        next_btn = wait.until(EC.element_to_be_clickable((
            By.XPATH, f"//a[contains(@class, 'hyY6CXtbcn') and text()='<{page_number+1}>'"]
        )))
        smooth_scroll_to(next_btn)
        time.sleep(0.3)
        driver.execute_script("arguments[0].click();", next_btn)
        print(f"<{page_number+1}>페이지로 이동 완료")
        time.sleep(2)
    except:
```


03

프로젝트 수행 과정

02

데이터 전처리

- **이모지 제거**
 - (r'[^\W\Ws.,!?ㄱ-ㅎㅏ-ㅣ가-힣]', '', text) → 한글, 영어, 숫자, 공백, 기본 문장부호만 남기고 나머지 삭제
- **특수문자 제거**
 - (r'[_~^··☆★▶✓❤️]'+, '', text) → 불필요한 장식 기호 제거
- **반복 자음/모음 축소**
 - (r'(\W)\1{2,}', r'\1\1', text) → 같은 문자가 3번 이상 반복되면 2번만 남김
 - ex) ㅋㅋㅋㅋ → ㅋㅋ, ㅎㅎㅎㅎ → ㅎㅎ
- **줄바꿈 제거**
 - (r'\Wn', ' ', text) → 여러 줄 리뷰를 한 줄로 변환
- **너무 짧은 리뷰 제거**
 - len(text.strip()) < 5: → 글자 수 5미만 리뷰 제외

```
import re

def clean_review(text):
    if not isinstance(text, str):
        return "" # 안전장치

    # 1. 이모지 제거
    text = re.sub(r'[^\W\Ws.,!?ㄱ-ㅎㅏ-ㅣ가-힣]', '', text)

    # 2. 특수문자 제거 (기본적인 문장부호는 남김)
    text = re.sub(r'[_~^··☆★▶✓❤️]'+, '', text)

    # 3. 반복 자음/모음 제거
    text = re.sub(r'(\W)\1{2,}', r'\1\1', text) # ㅋㅋㅋㅋ → ㅋㅋ

    # 4. 줄바꿈 제거
    text = re.sub(r'\Wn', ' ', text)

    # 5. 너무 짧은 리뷰 제거
    if len(text.strip()) < 5:
        return None

    # 6. "더보기" 제거
    if text == "더보기":
        return None

    return text.strip()
```

03

프로젝트 수행 과정

02

데이터 전처리

- 전처리

- 원본 리뷰(review_list)를 순회하며 전처리
- 전처리 성공 시 → 날짜(date) + 정제된 리뷰(review) 새 딕셔너리에 저장
- cleaned_reviews → 최종적으로 리스트 생성

```
# 전처리된 리스트 생성
cleaned_reviews = []

for review in review_list:
    raw_text = review.get("review", "")
    cleaned = clean_review(raw_text)
    if cleaned:
        cleaned_reviews.append({
            "date": review.get("date", ""),
            "review": cleaned
        })

print(f"전처리 완료! 유효한 리뷰 개수: {len(cleaned_reviews)}")
```

03

프로젝트 수행 과정

03

감성분석

- **Hugging Face 공개 모델 활용**
 - KoELECTRA → 한국어 리뷰 감성 분석에 특화된 Transformer 모델
 - 토큰라이저로 텍스트를 토큰 단위로 변환
- **감성 분석 함수(predict_sentiment)**
 - 입력 문장을 토큰화 후 모델에 전달
 - Softmax로 확률 계산 → 최종 레이블(긍정/부정) 반환
- **감성 분석 실행**
 - 전처리된 리뷰 데이터 불러오기
 - 각 리뷰마다 predict_sentiment() 실행
 - 결과(긍정/부정, 확률) → 새로운 컬럼에 저장
- **결과 저장**
 - “naver_reviews_sentiment.csv” → 분석 결과 저장

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification
import torch
import torch.nn.functional as F
import pandas as pd
from tqdm import tqdm

# 1. 모델 & 토큰라이저 로드
MODEL_NAME = "Copycats/koelectra-base-v3-generalized-sentiment-analysis"
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
model = AutoModelForSequenceClassification.from_pretrained(MODEL_NAME)
model.eval()

# 2. 레이블 매핑 (모델 문서 기준)
# 문서에 따르면: label 0 = negative (부정), label 1 = positive (긍정) :contentRe
{id2label = {0: "부정", 1: "긍정"}}

# 3. 감성 분석 함수
def predict_sentiment(text):
    inputs = tokenizer(text, return_tensors="pt", truncation=True, padding=True)
    with torch.no_grad():
        outputs = model(**inputs)
        probs = F.softmax(outputs.logits, dim=1)
        pred = torch.argmax(probs, dim=1).item()
    return id2label[pred], probs.squeeze().tolist()

# 4. 리뷰 데이터 불러오기
df = pd.read_csv("naver_reviews_cleaned.csv").dropna()

# 5. 감성 분석 실행
sentiments = []
probabilities = []

for review in tqdm(df['review']):
    try:
        label, probs = predict_sentiment(review)
    except Exception as e:
        label, probs = "분석 실패", [0.0, 0.0]
    sentiments.append(label)
    probabilities.append(probs)

df["sentiment"] = sentiments
df["probabilities"] = probabilities

# 6. 결과 저장
df.to_csv("naver_reviews_sentiment.csv", index=False, encoding="utf-8-sig")
print("감성분석 완료. 저장된 리뷰 수:", len(df))
print(df.head())
```

03

프로젝트 수행 과정

04

시각화

- 감성 분포 시각화

- 리뷰 개수 기준 긍정/부정 비율 계산
- 감성 분포를 막대그래프·파이차트 시각화
 - 막대그래프 → 리뷰 수 비교
 - 파이차트 → 전체 비율 시각화

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from wordcloud import WordCloud
```

```
# 한글 폰트 설정
```

```
plt.rcParams['font.family'] = 'Malgun Gothic'
plt.rcParams['axes.unicode_minus'] = False
```

```
# 결과 데이터 불러오기
```

```
df = pd.read_csv("naver_reviews_sentiment.csv")
```

```
# 감성 분포 계산
```

```
sentiment_counts = df['sentiment'].value_counts()
```

```
# 막대그래프
```

```
plt.figure(figsize=(6, 4))
```

```
sns.barplot(x=sentiment_counts.index, y=sentiment_counts.values)
```

```
plt.title("리뷰 감성 분포")
```

```
plt.xlabel("감성")
```

```
plt.ylabel("리뷰 수")
```

```
plt.show()
```

```
# 파이차트
```

```
plt.figure(figsize=(5, 5))
```

```
plt.pie(sentiment_counts.values, labels=sentiment_counts.index,
        colors=['skyblue', 'lightcoral'])
```

```
plt.title("감성 분석 비율")
```

```
plt.axis('equal')
```

```
plt.show()
```


03

프로젝트 수행 과정

04

시각화

- 긍정 리뷰 WordCloud
 - 긍정 리뷰만 모아 단어 빈도 분석
- 부정 리뷰 WordCloud
 - 부정 리뷰만 모아 단어 빈도 분석

```
# 데이터 불러오기
df = pd.read_csv("naver_reviews_sentiment.csv")

# Windows용 한글 폰트 경로 지정 (예: 맑은 고딕)
font_path = "C:/Windows/Fonts/malgun.ttf"

# 긍정 리뷰만 추출하여 하나의 문자열로 결합
positive_text = ' '.join(df[df['sentiment'] == '긍정']['review'])

# 워드클라우드 생성
wordcloud_pos = WordCloud(
    font_path=font_path,
    background_color='white',
    width=800,
    height=400
).generate(positive_text)

# 부정 리뷰 워드클라우드 생성
negative_text = ' '.join(df[df['sentiment'] == '부정'])

wordcloud_neg = WordCloud(
    font_path=font_path,
    background_color='black',
    width=800,
    height=400,
    colormap='Reds'
).generate(negative_text)

plt.figure(figsize=(10, 5))
plt.imshow(wordcloud_neg, interpolation='bilinear')
plt.axis('off')
plt.title("부정 리뷰 WordCloud", fontsize=16, color='w')
plt.show()
```

03

프로젝트 수행 과정

04

시각화

- 날짜 데이터 전처리
 - 문자열 형태의 날짜(25.09.29) → datetime 형식 변환
 - 결측치 drop → 변환 실패 데이터 제거
- 일별 리뷰 개수 집계
 - 날짜 단위로 리뷰 개수 집계
 - 특정 날짜별 리뷰 증가/감소 확인 가능
- 리뷰 추세 시각화
 - 라인 그래프를 통해 시간에 따른 리뷰 패턴 시각화
 - 특정 기간에 리뷰 급증

```
import pandas as pd
import matplotlib.pyplot as plt

# CSV 불러오기
df = pd.read_csv("naver_reviews_sentiment.csv")

# 날짜 컬럼 전처리
# 문자열 변환 + 공백 제거
df['date'] = df['date'].astype(str).str.strip()

# 날짜 변환 ("25.09.29." 같은 형식 처리)
df['date'] = pd.to_datetime(df['date'], format="%y.%m.%d.", errors='coerce')

# 변환 확인
print("변환된 날짜 예시:")
print(df['date'].head(20))
print("변환 실패 개수:", df['date'].isna().sum())

# 결측치 제거
df = df.dropna(subset=['date'])

# 한글 폰트 설정 (Windows: 맑은 고딕)
plt.rc('font', family='Malgun Gothic')
plt.rc('axes', unicode_minus=False)

# 일별 리뷰 개수 집계
daily_counts = df.groupby(df['date']).size()

# 시각화
plt.figure(figsize=(12,6))
plt.plot(daily_counts.index, daily_counts.values, marker='o')
plt.title("일별 리뷰 개수 추세")
plt.xlabel("날짜")
plt.ylabel("리뷰 개수")
plt.grid(True)
plt.show()
```

03

프로젝트 수행 과정

04

시각화

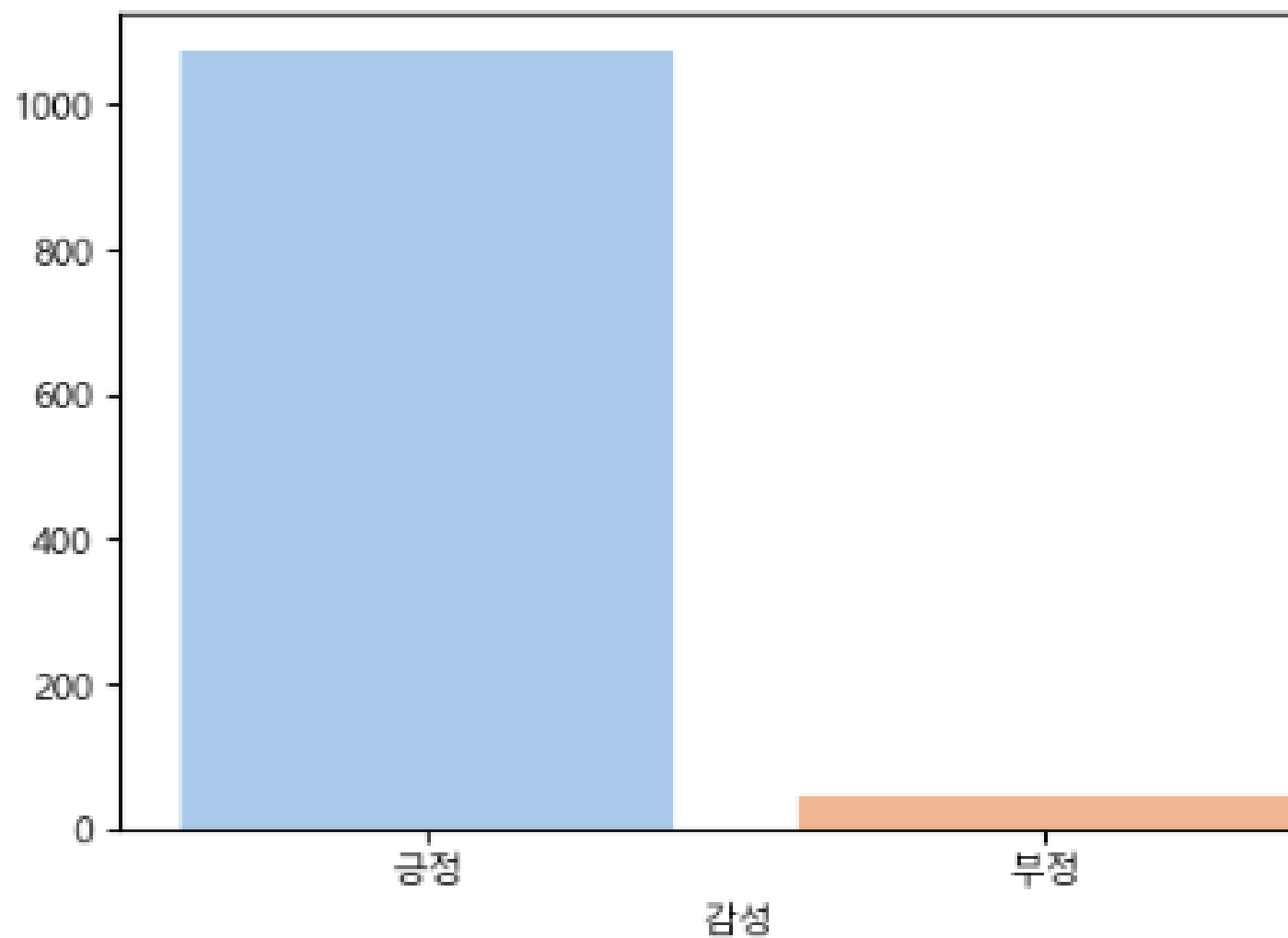
- **주간/월간 집계**
 - pd.Grouper 활용 → 주 단위(W), 월 단위(M) 그룹화
- **리뷰 추세 시각화**
 - 주간과 월간 리뷰 추세를 동시 비교 시각화
 - 리뷰 증가/감소 시점 → 이벤트, 프로모션 효과 확인 가능

```
# 3. 주간 / 월간 리뷰 추세
weekly_counts = df.groupby(pd.Grouper(key='date', freq='W')).size()
monthly_counts = df.groupby(pd.Grouper(key='date', freq='M')).size()

plt.figure(figsize=(12,6))
plt.plot(weekly_counts.index, weekly_counts.values, marker='o', label="주간")
plt.plot(monthly_counts.index, monthly_counts.values, marker='s', label="월간")
plt.title("주간 & 월간 리뷰 추세")
plt.xlabel("날짜")
plt.ylabel("리뷰 개수")
plt.legend()
plt.grid(True)
plt.show()
```

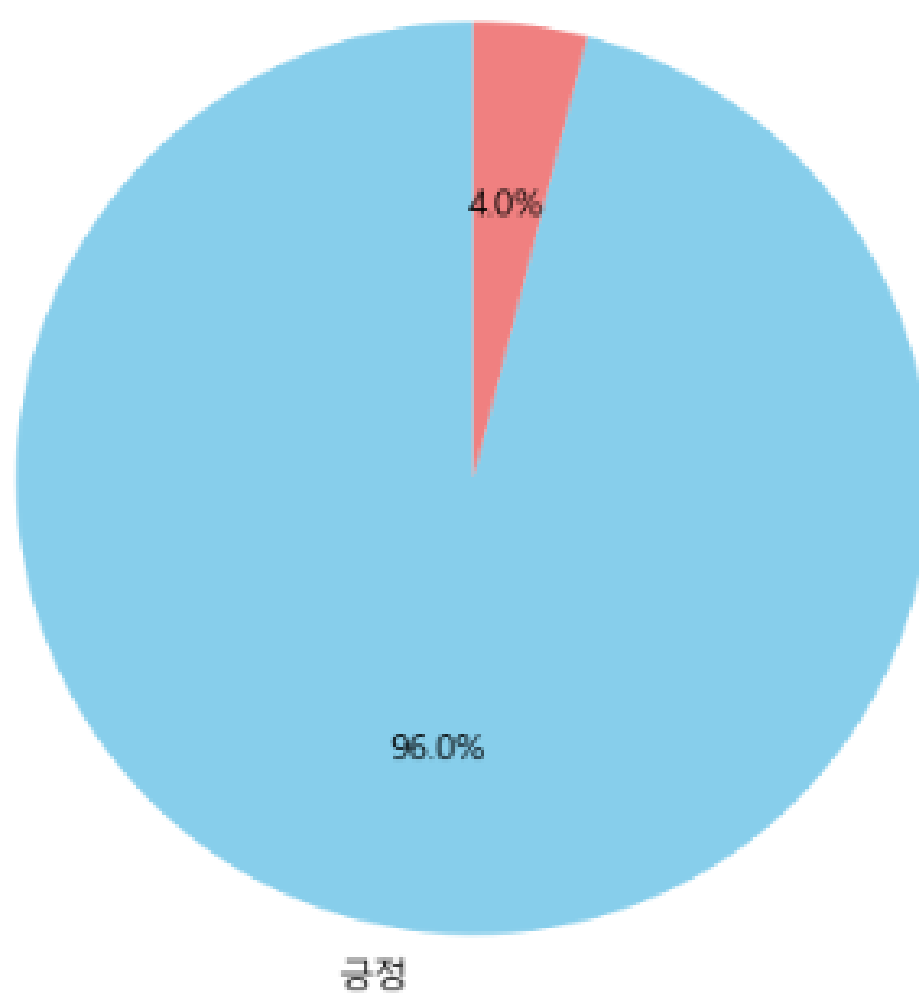
04 결과

리뷰 감성 분포



리뷰 감성 막대그래프

감성 분석 비율



리뷰 감성 파이차트

결과 해석

• 소비자 만족도

- 전체 리뷰의 대부분이 긍정적(96%)으로 나타나 제품/서비스에 대한 전반적인 만족도가 매우 높음을 의미

• 부정 리뷰의 의미

- 부정 리뷰는 4%에 불과하지만, 소수 의견 속에 개선 포인트가 숨어 있음

• 비즈니스 인사이트

○ 강점

- 긍정 리뷰가 많아 마케팅 활용 가능성 큼

○ 개선점

- 부정 리뷰를 정성적으로 분석해 고객 불만 원인 파악 및 대응 필요

04 결과

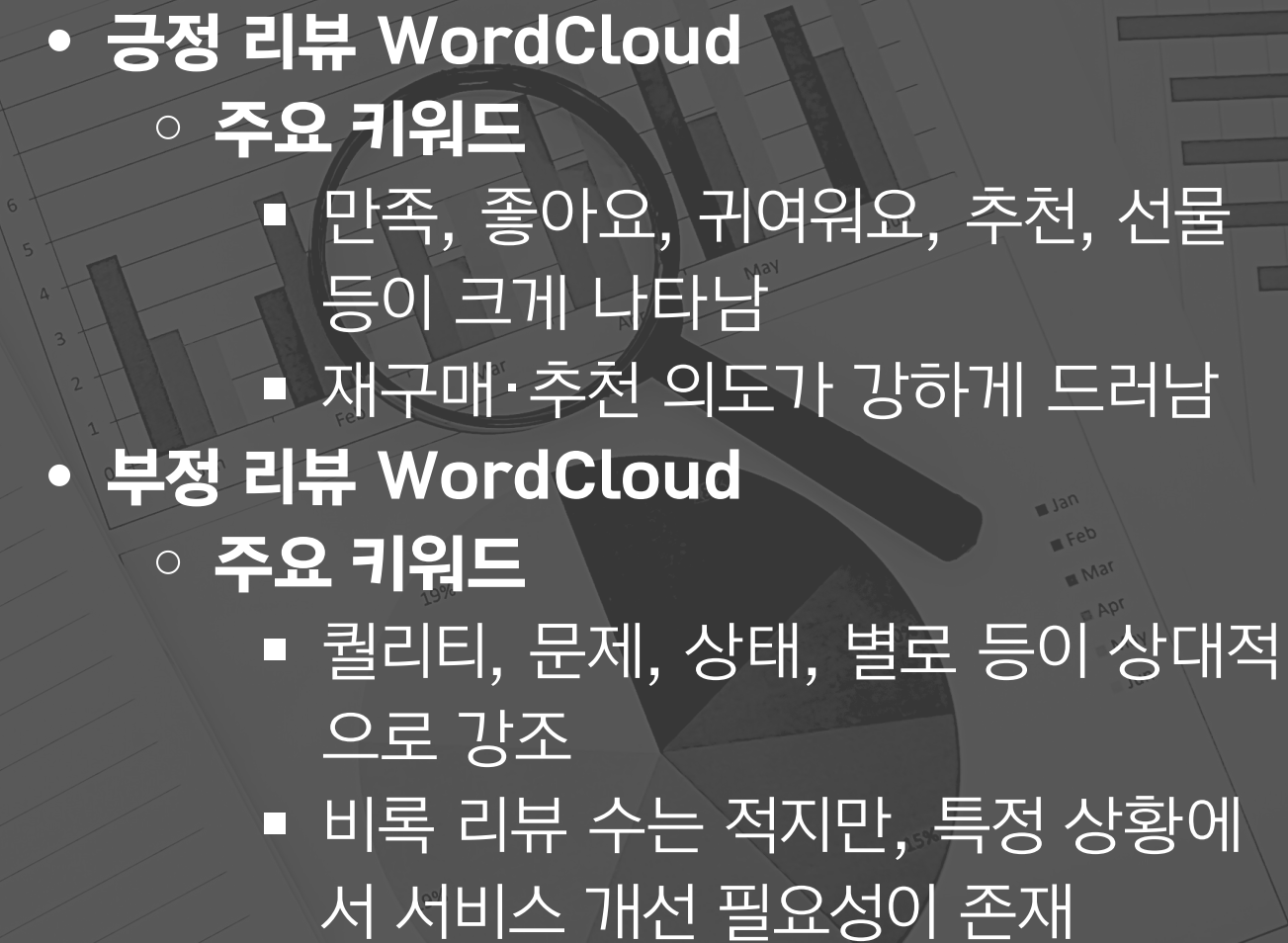


긍정 리뷰 WordCloud

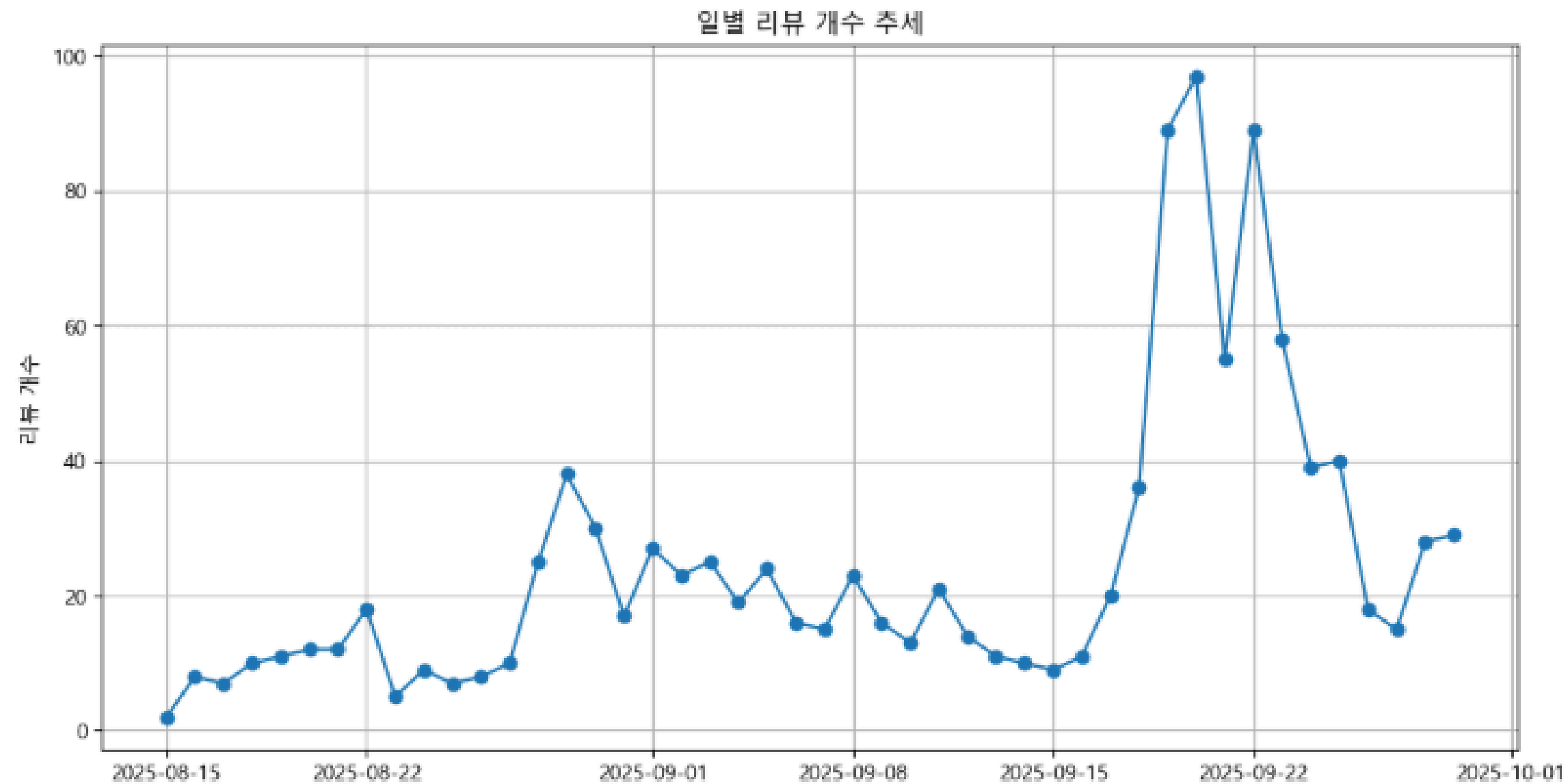


부정 리뷰 WordCloud

결과 해석

- 
- **긍정 리뷰 WordCloud**
 - **주요 키워드**
 - 만족, 좋아요, 귀여워요, 추천, 선물 등이 크게 나타남
 - 재구매·추천 의도가 강하게 드러남
 - **부정 리뷰 WordCloud**
 - **주요 키워드**
 - 퀄리티, 문제, 상태, 별로 등이 상대적으로 강조
 - 비록 리뷰 수는 적지만, 특정 상황에서 서비스 개선 필요성이 존재

04 결과

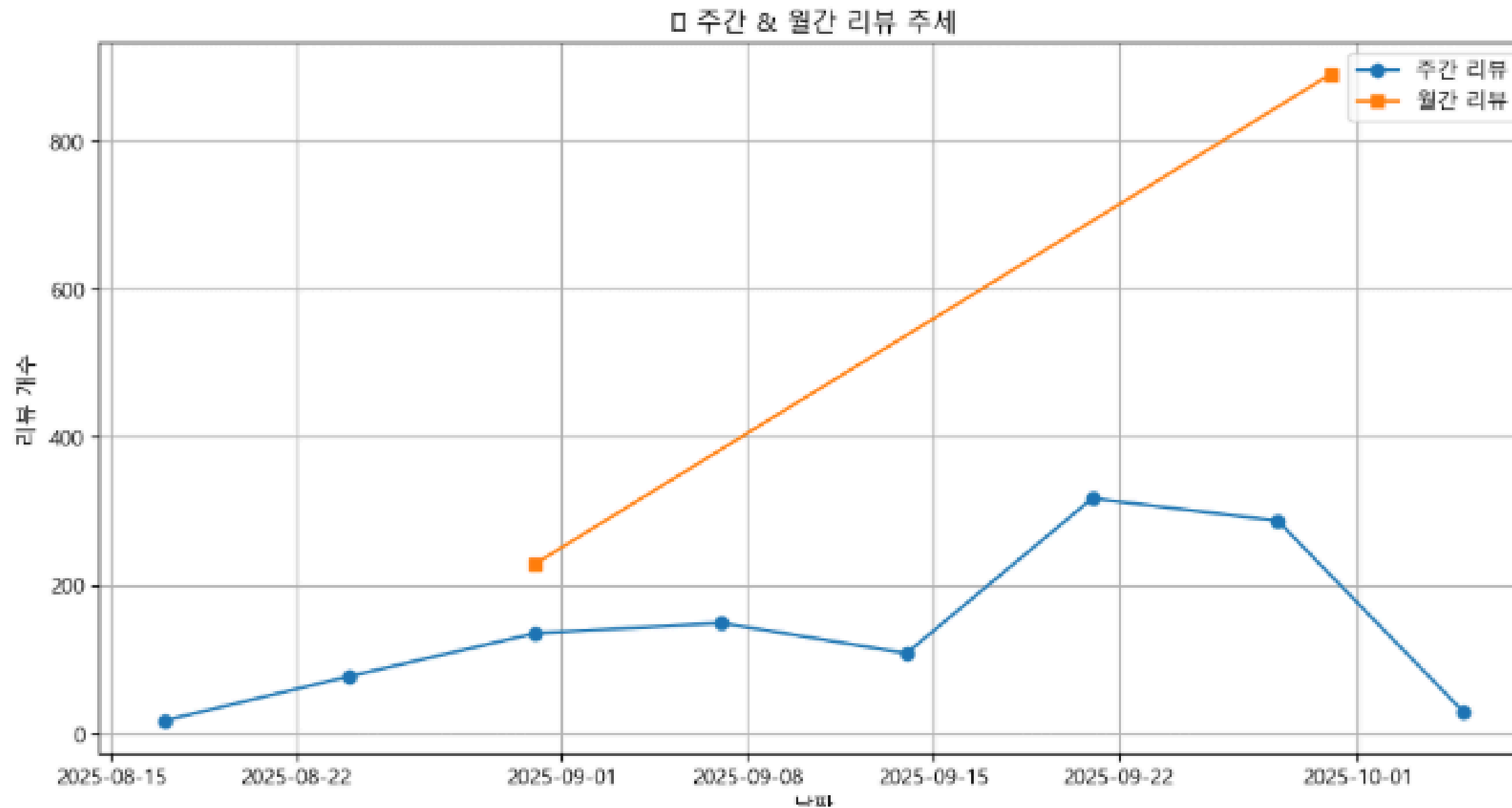


시계열/추세 분석 그래프

결과 해석

- 그래프에서 보이는 리뷰 개수 추세는 시간에 따라 일정한 패턴 없이 특정 시점에 급격히 증가하는 양상을 보임
- 특히 9월20일 전후에 리뷰가 폭발적으로 증가하며 최대 100경에 가까운 리뷰가 기록됨
- 이후 급격히 감소하다가 다시 단기적인 상승/하락을 반복
- 특정 이벤트(프로모션, 할인 행사, 광고, 제품 출시/업데이트)와 연관이 있을 가능성이 큼
- 전체적으로 보면 안정적이지 않고 이벤트성 요인에 크게 영향을 받는 데이터 분포라고 볼 수 있음

04 결과



주간/월간 리뷰 분석 그래프

결과 해석

- 주간 리뷰 추세는 주마다 일정한 편차를 보이지만 9월 셋째 주를 기점으로 급격히 증가 후 다시 하락하는 모습 보임
- 월간 리뷰 추세는 8월 대비 9월이 크게 증가하여 급격한 상승 곡선을 그림
- **종합**
 - 월간 단위의 큰 증가세는 장기적으로 리뷰 활성화가 진행되고 있음을 의미
 - 주간 단위의 급등락은 특정 기간의 이벤트성 요인을 시사함

05

성과 및 배운점



성과

- 리뷰 데이터 수집 및 정제
 - 네이버 리뷰 데이터를 실제로 크롤링하고 전처리하여 분석 가능한 형태의 데이터셋 구축에 성공
 - 불필요한 텍스트 (이모지, 특수문자 등) 제거 및 정제 과정을 통해 데이터 품질 개선
- 감성 분석 모델 적용
 - 한국어 특화된 KoELECTRA 모델을 활용하여 리뷰의 긍정·부정 감성을 자동 분류
 - 전체 리뷰 중 긍정 리뷰가 96%, 부정 리뷰가 4%임을 정량적으로 확인
- 시각화 및 인사이트 도출
 - 감성 분포 그래프, 워드클라우드, 시계열/주간/월간 추세 그래프 등을 통해 데이터 패턴 시각화
 - 특정 기간에 리뷰가 폭발적으로 증가한 현상 발견
→ 이벤트/마케팅 요인 가능성 제시

05

성과 및 배운점



배운점

- **데이터 전처리의 중요성**
 - 특수문자, 짧은 리뷰 등 불필요한 데이터를 제거하지 않으면 분석 결과에 왜곡이 생김을 경험
 - 따라서 텍스트 정제와 클린 데이터 확보가 분석의 출발점임을 체감
- **AI 모델의 적용과 한계**
 - Transformer 기반 한국어 감성분석 모델을 실제 데이터에 적용하면서 AI 모델의 활용 가능성과 강점을 확인
 - 동시에 리뷰 데이터의 불균형(긍정이 과도하게 많음)이 결과 해석에 영향을 줄 수 있음을 배움
- **시각화를 통한 인사이트 발견**
 - 단순 수치만 보는 것보다 그래프와 워드클라우드로 표현했을 때 이벤트성 요인, 긍정·부정 키워드 차이 등 중요한 통찰을 얻을 수 있었다
 - 데이터 분석은 숫자 → 그림 → 의미 해석의 과정이 필수적임을 배움
- **이 프로젝트에서 배운 가장 큰 점은 데이터 전처리와 시각화를 통한 인사이트 도출의 중요성입니다.**

06

추가 사항

01

프로젝트 한계

데이터의 불균형

리뷰 수집 범위 제한

감성 분류 단순화

02

개선 및 확장 방향

데이터 다양화

감정 세분화

모델 개선

실시간 분석

03

활용 방안
(실무 연계성)

마케팅 전략 수립

고객 만족도 관리

제품 개선 피드백

THANK YOU

CONTACT.

임현성

010-8905-7711

qkek033@gmail.com